Results: what we measured and what it means

Thesis: *A multimodal framework for failure detection, adaptive recovery, and experience memory in autonomous web agents*. Prepared 2026-10-06 for the supervisor meeting; updated 2026-10-14 with the second-agent results (Tables 12–20).

Our system has two parts. **Task 1** asks: *can our trained model tell whether a web-agent step failed?* **Task 2** asks: *if we put that model on top of a real agent, does the agent finish more tasks?* The framework is called **WebFARM** (Failure-Aware Recovery and Memory): P1 failure detection, adaptive recovery and experience memory on top of an unmodified open-source agent. Task 1 results are final. Task 2 results come from three completed paired runs on the same 42 real-website tasks × 2 repeats: *web-v3* (4–6 October 2026: Browser Use with and without WebFARM, 168 episodes), *web-v4* (11–13 October: the raw SeeAct agent, 84 episodes) and *web-v5* (14 October: SeeAct + WebFARM, 84 episodes). Every run uses the same frozen actor, **Qwen2.5-VL-7B-Instruct**, the same budgets and the same success rule.

Table 1. Which backbone to use (trained on our data, full validation set)

Backbone	Step failure (MCC)	Step failure (Macro-F1)	Recovery worked? (MCC)	Recovery worked? (acc.)	Worth remembering? (MCC)
Qwen2-VL-2B	0.624	0.809	0.796	90.9 %	0.663
Qwen2.5-VL-7B (selected)	**0.678**	**0.839**	**0.852**	**93.5 %**	**0.695**
InternVL3.5-8B-HF	0.641	0.820	0.840	92.9 %	0.654

7,861 validation steps; 1,858 recovery cases; seed 42; one checkpoint per model, chosen by a rule fixed in advance.

What this means. All three models learn the job well from our data (MCC 0.62–0.68 on step failure; 0.80–0.85 on recovery). Qwen2.5-VL-7B is the best on every column, so it is the model we use in the agent. The gap between models is small, which is good news: our dataset works for different backbones, not just one.

How good is good? MCC runs from –1 to +1; 0 means guessing. Values around 0.65 mean the model gets most steps right in both directions (it catches real failures *and* does not cry wolf on good steps). Recovery checking is stronger (0.85): "did the fix work?" is an easier question than "did this step fail?".

Limit. These are single-seed validation numbers. The locked test split has not been used yet.

Table 2. Does training matter? Same model, same cases, trained vs. only prompted

Frozen subset of our validation data: 240 steps and 120 recovery cases. The "prompted" rows use the **same untrained model** and ask it to judge the step the way three public agents do (their self-check instructions, adapted). Unreadable answers count as wrong.

2a. "Did this step fail?" (240 cases)

How the step is judged	Model	Trained on our data?	MCC	Macro-F1	Correct on all cases	Readable answer
Browser Use-style prompt	Qwen2.5-VL-7B	no	0.000	0.237	11 %	87 / 240
Agent S2-style prompt	Qwen2.5-VL-7B	no	0.000	0.183	16 %	170 / 240
WebVoyager-style prompt	Qwen2.5-VL-7B	no	—	—	0 %	0 / 240
Browser Use-style prompt	Qwen2-VL-2B	no	0.000	0.206	25 %	228 / 240
Agent S2-style prompt	Qwen2-VL-2B	no	0.000	0.199	25 %	238 / 240
WebVoyager-style prompt	Qwen2-VL-2B	no	0.000	0.200	23 %	220 / 240
Our trained heads	InternVL3.5-8B	yes	0.558	0.768	80 %	240 / 240
Our trained heads	**Qwen2.5-VL-7B**	**yes**	**0.649**	**0.820**	**85 %**	**240 / 240**

2b. "Did the recovery work?" (120 cases)

How it is judged	Model	Trained?	MCC	Macro-F1	Correct on all cases	Readable answer
Browser Use-style prompt	Qwen2.5-VL-7B	no	–0.122	0.400	17 %	30 / 120
Agent S2-style prompt	Qwen2.5-VL-7B	no	–0.104	0.331	38 %	91 / 120
WebVoyager-style prompt	Qwen2.5-VL-7B	no	—	—	0 %	0 / 120
Browser Use-style prompt	Qwen2-VL-2B	no	0.000	0.329	46 %	112 / 120
Agent S2-style prompt	Qwen2-VL-2B	no	0.000	0.333	50 %	120 / 120
WebVoyager-style prompt	Qwen2-VL-2B	no	0.000	0.335	46 %	109 / 120
Our trained heads	InternVL3.5-8B	yes	0.884	0.942	94 %	120 / 120
Our trained heads	**Qwen2.5-VL-7B**	**yes**	**0.841**	**0.916**	**92 %**	**120 / 120**

What this means. This is the most important table for Task 1. Every prompted row sits at **MCC 0** (pure guessing), and the prompted models often cannot even give a readable yes/no (WebVoyager-style: zero readable answers out of 240). The **same** Qwen2.5-VL-7B, after training on our data, jumps to **MCC 0.65** with a readable answer on every case. Same model, same cases, only training differs. So the improvement comes from **our dataset and training**, not from the size or brand of the model.

Why this is good news, not bad. If the untrained model had already scored well, our dataset would add nothing. The fact that it scores zero is what makes the training (and the dataset) necessary. It also explains why plain agents loop: their own self-check ("did my last step work?") is no better than a coin flip.

One limit to state. The prompted rows copy those agents' *self-check instructions*; they are not the full agents running live. So the claim is "prompted self-assessment does not work", not "those agents are bad".

Small note. Table 2 numbers (0.649 / 0.841) differ a little from Table 1 (0.678 / 0.852) because Table 2 uses 240 / 120 selected cases and Table 1 uses the full validation set. Both are reported as what they are.

Table 3. Our detector next to published detectors (context only; different data)

Model	Size	Trained for this?	Data it was tested on	Accuracy
GPT-4o, best prompt [1]	undisclosed	no	failure traces of [1]	83.8 %
InternVL2-76B, best prompt [1]	76 B	no	same	82.4 %
Gemini-1.5-pro, best prompt [1]	undisclosed	no	same	79.7 %
GPT-4o, default prompt [1]	undisclosed	no	same	74.4 %
Gemini-1.5-flash, default prompt [1]	undisclosed	no	same	61.5 %
Ours: Qwen2.5-VL-7B + trained heads	**7 B, 4-bit**	**yes**	**our validation set**	**≈83 % (MCC 0.678)**

What this means. A 7B open model trained on our data lands in the same range as prompted GPT-4o and a 76B model, at about a tenth of the size and running locally. **But** these are different test sets, so this table is background, not a race. The honest sentence is "comparable magnitude on different data". Never write "beats GPT-4o".

Table 4. Agent level: does adding failure handling help a real agent?

System	What is added to the base agent	Actor model	Benchmark	Base agent	With system	Gain
Multimodal Auto Validation [2]	vision validator + retry	GPT-4-class	WebVoyager	76.2	81.2	+5.0
BacktrackAgent [3]	error detection + backtracking	Qwen2-VL (fine-tuned)	Auto-UI (mobile)	21.6	29.7	+8.1
WebCoach [4]	memory coach	Qwen-VL-7B	WebVoyager	32.8	28.8–31.1	–1.7 to –4.0
WebCoach [4]	memory coach	Qwen-VL-32B	WebVoyager	49.5	54.7–57.1	+5 to +8
Devil's Advocate [5]	reflection before acting	GPT-4	WebArena	19.8	23.5	+3.7
OSCAR [6]	re-planning on state change	GPT-4	GAIA	22.4	28.7	+6.3
SkillWeaver [7]	learned skills	GPT-4o	WebArena	22.6	29.8	+7.2
WebFARM on Browser Use, development tasks (20)	**trained failure detection + recovery + memory**	**Browser Use + Qwen2.5-VL-7B base**	**real websites, 40 pairs**	**70.0**	**87.5**	**+17.5**
WebFARM on Browser Use, held-out tasks (22)	**same**	**same**	**real websites, 44 pairs**	**75.0**	**84.1**	**+9.1**
WebFARM on Browser Use, all tasks (42)	**same**	**same**	**real websites, 84 pairs**	**72.6**	**85.7**	**+13.1**
WebFARM on SeeAct, development tasks (20)	**same**	**SeeAct + Qwen2.5-VL-7B base**	**real websites, 40 pairs**	**60.0**	**77.5**	**+17.5**
WebFARM on SeeAct, held-out tasks (22)	**same**	**same**	**real websites, 44 pairs**	**63.6**	**86.4**	**+22.7**
WebFARM on SeeAct, all tasks (42)	**same**	**same**	**real websites, 84 pairs**	**61.9**	**82.1**	**+20.2**

What this means. Published systems that add checking or recovery to an agent usually gain **+4 to +8 points**. WebFARM gains +13.1 points on Browser Use and +20.2 on SeeAct, with +9.1 and +22.7 on tasks never seen during design, so it sits above that range, with a 7B open actor rather than GPT-4, and it is the only system in the table shown on two different base agents. These are different benchmarks, so the comparison is of relative gains, not absolute rates.

The WebCoach 7B row is a warning that supports us. With a 7B actor, extra advice made the agent *worse*; it only helped bigger actors. We saw exactly this in our first run (web-v2: ours 20/38 vs. baseline 27/38) and redesigned the recovery layer (v3) to remove it. The final runs (Tables 6–8 and 16) show that it did.

Two things in our evaluation that none of the cited papers do: **paired comparison with a significance test**, and a **held-out task set** that was never looked at while designing the system.

Table 5. Why the base agent fails (our traces vs. published analyses)

Source	Main failure causes
AutoWebGLM [9]	hallucination 44 %, poor visual recognition 28 %, misread task 20 %, pop-ups 8 %
WebSuite [10]	simple actions (click, type) succeed 76–85 %; "find/filter" tasks only 40–44 %
Ours: Browser Use (web-v2 baseline, 11 failures)	**repeating a useless action 9 (82 %): scrolling past the page end 5, clicking/typing the same box 4; false "done" 1; broken output 1**
Ours: raw SeeAct (web-v4, 32 failures)	**click loops: 85 % of actions in failed episodes are clicks, 19 of 32 failures contain a run of 5 or more identical actions, 28 of 32 run out of the 15-step budget; 56 clicks over all 84 episodes hit hidden or stale elements and were rejected by the browser; 0 unparsable answers, 0 token-limit hits**

What this means. Our base agent's main problem is that it keeps doing something that has no effect while believing it worked (the "hallucination" class in [9]). That is exactly what a before/after failure detector can see, which is why our approach targets it. The two rare cases (declaring "done" without acting; malformed output) produce no step to judge and are outside what any step-level detector can fix. SeeAct fails differently (click loops instead of scroll loops) but for the same underlying reason: it cannot tell that its last action changed nothing. Both failure styles are visible to a before/after detector, which is why one framework helps both agents.

Task 2 results: Browser Use with and without WebFARM

Setup. 42 real-website tasks from domains in our dataset: 20 *development* tasks (inspected while the recovery layer was designed) and 22 *held-out* tasks (written and verified before any run, never inspected). Each task was run twice by each system, back-to-back, giving 84 pairs. The base agent is **Browser Use 0.13.10 driven by the frozen Qwen2.5-VL-7B-Instruct**. "Browser Use + WebFARM" is the same agent, prompts, 15-step budget and start pages, with our framework attached (P1 failure detection, adaptive recovery, experience memory). A task counts as completed when the environment sees the target page URL; the agent never sees this rule.

Table 6. Task completion of Browser Use with and without WebFARM

Task set	Pairs	Browser Use	Browser Use + WebFARM	Δ (pts)	p
Development	40	70.0	87.5	+17.5	0.039
Held-out	44	75.0	84.1	+9.1	0.219
All	**84**	**72.6**	**85.7**	**+13.1**	**0.007**

Completion rate in %. Actor: Qwen2.5-VL-7B-Instruct (frozen) in both columns. p: exact two-sided sign test on discordant pairs.

What this means. This table answers the main question of Task 2: does the framework make Browser Use finish more tasks? Over all 84 paired episodes, Browser Use alone completed 72.6 % of tasks; with our framework it completed 85.7 %, a gain of 13.1 points. Out of every 100 tasks Browser Use would fail on about 27; with our framework it fails on about 14, so **roughly half of its failures are removed**. The result is statistically significant (p = 0.007). On the development tasks the gain is +17.5. On the held-out tasks, never inspected during design, the gain is +9.1 and not significant on its own, because only six pairs differ. The direction is the same on tasks the system had never seen, so the improvement is not an artefact of tuning to known tasks.

Table 7. Paired outcomes

Task set	Helped	Hurt	Same
Development	8	1	31
Held-out	5	1	38
All	**13**	**2**	**69**

Helped: Browser Use failed, Browser Use + WebFARM completed. Hurt: the reverse.

What this means. Each task is run by both systems back-to-back, so pairs can be read directly. In 69 of 84 pairs the result was the same, as expected: the framework does nothing unless the failure detector fires. Among the 15 pairs that differed, ours won 13 and lost 2. **When the framework changes Browser Use's outcome, it improves it in 87 % of cases.** The two losses (Wikipedia "black hole", jQuery API) are examined in the error analysis.

Table 8. Rescue and harm rates

Task set	Rescue rate	Harm rate
Development	67 (8/12)	4 (1/28)
Held-out	45 (5/11)	3 (1/33)
All	**57 (13/23)**	**3 (2/61)**

Rescue: share of Browser Use failures completed with WebFARM. Harm: share of Browser Use successes failed with WebFARM. Values in %.

What this means. A recovery system must fix failures and leave successes alone. The framework rescued 57 % of the episodes Browser Use failed (67 % development, 45 % held-out) while disturbing only 3 % of the episodes Browser Use would have completed. The low harm rate matters as much as the rescue rate: our first design (web-v2) had a 22 % harm rate and 0 % rescue, because it slowed the actor and sent misleading labels. The redesigned layer removed almost all harm while adding the rescues.

Table 9. Cost per episode

Measure	Browser Use	Browser Use + WebFARM
Mean time (s)	220	175
Mean steps, both completed (n = 59)	3.08	3.32

What this means. The framework adds work per step (an assessment after each action; extra actor calls after a detected failure), yet the average episode is **20 % shorter**. When Browser Use fails it usually spends all 15 steps repeating a useless action; ours breaks the loop or finishes. On the 59 pairs both completed, ours used 0.24 steps more on average, the price of occasional recovery detours. The framework costs almost nothing when Browser Use is doing well and saves time when it is not.

Table 10. Recovery activity of WebFARM on top of Browser Use (84 episodes)

Measure	Count
Episodes with at least one recovery	31
Recovery attempts	55
Option selections by the actor	69
Repeat-guard blocks	30
Uncertain alarms suppressed (P < 0.9)	22
Memory records written	34
Memory records shown	10

What this means. The failure detector opened a recovery in 31 of 84 episodes; in the other 53 it stayed silent and Browser Use's input was identical to the baseline's. When it fired, the actor picked an offered option 69 times and was stopped from repeating a failed action 30 times, which is where the loop-breaking comes from. The confidence gate held back 22 uncertain alarms, protecting the harm rate. The memory stored 34 experiences and showed 10 on repeat visits; its effect is small here because each site is visited only twice.

Table 11. Per-task outcome of all four systems (two repeats; Y completed, N failed)

Task	Split	Browser Use	Browser Use + WebFARM	SeeAct	SeeAct + WebFARM
Wikipedia: Alan Turing	Dev	N Y	Y Y	Y Y	Y Y
Wikipedia: Eiffel Tower	Dev	N Y	Y Y	Y Y	Y Y
arXiv: cs.AI list	Dev	Y Y	Y Y	N N	Y Y
Debian: security	Dev	Y Y	Y Y	N Y	N N
Creative Commons: licenses	Dev	Y Y	Y Y	N N	N N
GOV.UK: renew passport	Dev	N Y	Y Y	N N	N Y
GOV.UK: bank holidays	Dev	N Y	Y Y	N N	N Y
TED: talks	Dev	N N	N Y	Y Y	Y Y
jQuery: API docs	Dev	Y Y	Y N	Y Y	Y Y
EPA: climate change	Dev	N N	Y Y	N Y	Y Y
IETF: RFCs	Dev	N N	Y N	N N	Y Y
Europa: EU countries	Dev	Y Y	Y Y	Y Y	Y N
arXiv: math list	Dev	Y Y	Y Y	N N	Y N
Debian: news	Dev	N N	N N	N N	Y N
6 other tasks	Dev	Y Y	Y Y	Y Y	Y Y
cPanel: pricing	Held-out	N N	Y Y	Y Y	Y Y
EPA: laws & regulations	Held-out	Y Y	Y Y	N Y	Y Y
EPA: recycling	Held-out	N N	N N	N N	N Y
GitLab: pricing	Held-out	Y Y	Y Y	Y Y	Y N
GOV.UK: driving licence	Held-out	N N	N Y	N N	Y Y
GOV.UK: state pension age	Held-out	Y Y	Y Y	N N	Y Y
IETF: about	Held-out	Y Y	Y Y	N N	N N
IETF: meetings	Held-out	Y Y	Y Y	N Y	Y Y
Nextcloud: install	Held-out	Y Y	Y Y	N N	Y Y
NOAA: weather	Held-out	N N	N N	N N	Y Y
Wikipedia: community portal	Held-out	N Y	Y Y	Y N	Y Y
Wikipedia: black hole	Held-out	Y N	N N	Y N	N N
Wikipedia: Marie Curie	Held-out	N Y	Y Y	Y Y	Y Y
9 other tasks	Held-out	Y Y	Y Y	Y Y	Y Y

Tasks where all four systems completed both repeats are collapsed into the "other tasks" rows.

What this means. Three patterns appear. **Consistent rescues**: on EPA climate change and cPanel pricing Browser Use failed both times and Browser Use + WebFARM succeeded both times; on GOV.UK state pension age, IETF meetings, Nextcloud and NOAA weather raw SeeAct failed both times and SeeAct + WebFARM succeeded both times, so these are not chance. **Variance rescues**: where a raw agent failed once and succeeded once, the agent with WebFARM usually succeeded both times; the framework removes the agents' bad days. **Beyond the framework**: EPA recycle r0, Debian news r1 and black hole r1 failed in all four systems; on these pages the target link shares no words with the goal, or the site search misleads. The two agents fail on different tasks (SeeAct on list and policy pages, Browser Use on search-driven pages), yet the same framework fixes most failures of both.

Task 2 (continued): the same WebFARM on a second agent, SeeAct

Setup. SeeAct [11] (OSU NLP, ICML 2024) is an unmodified open-source web agent with a different design from Browser Use: per step it lists the interactive elements inside the viewport, draws numbered boxes on the screenshot, asks the model to reason (turn 0) and then to pick an element letter plus ACTION / VALUE (turn 1), and executes with Playwright, two model calls per step. It ran on the same 42 tasks × 2 repeats, the same headless browser, start pages, 15-step and 2700 s budgets, the same frozen Qwen2.5-VL-7B-Instruct actor and the same success rule. "SeeAct + WebFARM" attaches the framework through a thin adapter: the assessor, trigger gate (0.9), recovery note, page facts, numbered options, repeat guard and memory are the same code and settings as on Browser Use; only how the note enters SeeAct's prompt, how an element is identified and how a chosen option is executed (as a native SeeAct GOTO / TYPE / PRESS ENTER / GO BACK) are agent-specific.

Table 12. Task completion of all four systems on the same 84 paired episodes

System	Completed	Rate	95 % CI (Wilson)
SeeAct (raw)	52 / 84	61.9 %	[51.2, 71.6]
SeeAct + WebFARM	**69 / 84**	**82.1 %**	**[72.6, 88.9]**
Browser Use (raw)	61 / 84	72.6 %	[62.3, 81.0]
Browser Use + WebFARM	**72 / 84**	**85.7 %**	**[76.7, 91.6]**

What this means. With the same 7B model and tasks, raw SeeAct completes fewer tasks than raw Browser Use (61.9 % vs 72.6 %). Adding WebFARM lifts SeeAct to 82.1 % and Browser Use to 85.7 %. The weaker agent with the framework (82.1 %) completes more tasks than the stronger agent without it (72.6 %). The confidence intervals of the two raw agents overlap, and so do those of the two agents with WebFARM; the intervals of "raw" and "with WebFARM" do not overlap for SeeAct.

Table 13. All paired comparisons (exact two-sided sign test on discordant pairs; 84 pairs each)

Comparison	First better	Second better	Same	p
Browser Use + WebFARM vs Browser Use	**13**	**2**	**69**	**0.007**
SeeAct + WebFARM vs SeeAct	**21**	**4**	**59**	**0.0009**
Browser Use + WebFARM vs SeeAct + WebFARM	11	8	65	0.65
Browser Use vs SeeAct (both raw)	18	9	57	0.12
SeeAct + WebFARM vs Browser Use (raw)	18	10	56	0.18

What this means. The two rows that matter are the first two: on each agent, adding WebFARM wins far more pairs than it loses, and both differences are significant. The other three rows say that the two raw agents are not significantly different from each other, and neither are the two agents once WebFARM is attached. In plain words: the framework matters more than which agent you start from.

Table 14. By task set, all four systems (completion in %)

Task set	n	SeeAct	SeeAct + WebFARM	Helped / hurt	p	Browser Use	Browser Use + WebFARM	Helped / hurt	p
Development	40	60.0	77.5	9 / 2	0.065	70.0	87.5	8 / 1	0.039
Held-out	44	63.6	86.4	12 / 2	0.013	75.0	84.1	5 / 1	0.219
All	**84**	**61.9**	**82.1**	**21 / 4**	**0.0009**	**72.6**	**85.7**	**13 / 2**	**0.007**

What this means. On SeeAct the gain is larger on the held-out tasks (+22.7, significant) than on the development tasks (+17.5). The recovery layer was designed while watching Browser Use on the development tasks and never saw SeeAct or the held-out tasks, so this is a genuine transfer result, not tuning. On Browser Use the pattern is the reverse (+17.5 development, +9.1 held-out), as reported in Table 6.

Table 15. Rescue and harm rates on both agents

Agent	Rescue rate (failures fixed)	Harm rate (successes lost)	Net pairs changed
Browser Use	57 % (13 / 23)	3 % (2 / 61)	+11 of 84
SeeAct	66 % (21 / 32)	8 % (4 / 52)	+17 of 84

What this means. WebFARM fixes about two thirds of SeeAct's failures and well over half of Browser Use's, while disturbing 8 % and 3 % of their successes. The higher harm rate on SeeAct comes from four pairs, examined in Table 20: two are interactions between the repeat guard and SeeAct's own stop rule or a scoring-rule URL variant, not wrong detections.

Table 16. Cost per episode, all four systems

Measure	SeeAct	SeeAct + WebFARM	Browser Use	Browser Use + WebFARM
Mean time per episode (s)	182	123	220	175
Model calls per step	2	2 (+ assessment, options)	1	1 (+ assessment, options)
Mean steps on pairs both variants completed	3.31 (n = 48)	3.40 (n = 48)	3.08 (n = 59)	3.32 (n = 59)

What this means. WebFARM makes both agents faster on average (SeeAct by 32 %, Browser Use by 20 %) because it ends the loops that otherwise run to the 15-step budget, while adding only 0.1–0.2 steps on the tasks the raw agent completes anyway. The extra model calls (an assessment after each action, an option prompt after a detected failure) are paid for by the loops avoided.

Table 17. Recovery activity of WebFARM on both agents (84 episodes each)

Measure	On Browser Use	On SeeAct
Episodes with at least one recovery	31	56
Recovery attempts	55	97
Steps assessed by P1	319	365
Confident failures (P ≥ 0.9; action blocked)	–	160
Uncertain alarms held back by the gate (P < 0.9)	22	31
Option prompts / options taken	86 / 69	155 / 143
Options taken: open goal link / search / Enter after search / scroll-top-then-search / go back	26 / 19 / 19 / 4 / 0	58 / 42 / 36 / 5 / 2
"0 = decide myself"	17	12
Repeat-guard blocks / overrides	30 / –	45 / 4
Memory records written / shown	34 / 10	81 / 15

What this means. WebFARM fires far more often on SeeAct (56 of 84 episodes) than on Browser Use (31) because SeeAct's typical failure, clicking the same wrong or hidden menu element, is detected as a confident failure on its first repetition. The same three options do the work on both agents: open a goal-matching link, search the site, press Enter. The actor took an offered option in 92 % of prompts on SeeAct and 80 % on Browser Use, so the options are acceptable to the actor on both.

Table 18. How raw SeeAct fails (32 failed episodes)

Observation	Count
Stop reason: 15-step budget exhausted	28 of 32
Stop reason: five consecutive no-ops / self-declared done	3 / 1
Episodes with a loop of 5 or more identical actions	19 of 32
Action mix in failed episodes	CLICK 85 %, TYPE 7 %, SELECT 3 %
Executions rejected by the browser (all 84 episodes)	56 (click timeout 30, fill on a non-input 17, select on a non-select 8)
Navigations outside the task's domains, undone	11
Unparsable answers / token-limit hits	0 / 0

What this means. SeeAct's dominant failure is repeated clicking of the same wrong element: a navigation-menu entry that does not lead to the target, or a listed element that is not clickable. It never produced an unreadable answer and never hit the token limit, so the 7B model follows SeeAct's format well; the failures are decision failures, the class WebFARM targets. Browser Use's failures were scroll loops (Table 5); the detector sees both because both leave the page unchanged.

Table 19. Stop reasons of SeeAct + WebFARM (84 episodes)

Outcome	Count
Completed (environment saw the target page)	69
Failed: SeeAct's own five-no-op rule ended the episode	8
Failed: 15-step budget	7
Infrastructure reruns (not counted; browser page closed after a navigation)	1

Table 20. The four pairs WebFARM lost on SeeAct

Pair	What happened	Class
GitLab pricing, repeat 2	SeeAct's first click on the hidden "Pricing" menu item timed out. Raw SeeAct retried it three times and then found the visible link. With WebFARM the rejected click was blocked, the actor answered with no valid element four times, and SeeAct's own five-no-op rule ended the episode at step 5.	repeat guard × agent stop rule
Debian security, repeat 2	WebFARM's link option took the agent to /security/index.en.html, the same page as the target /security/; the exact-page rule scores it as a different page. No other run ever observed this URL variant.	scoring-rule URL variant
Wikipedia black hole, repeat 1	The search query built from the goal was "box article black holes" ("search box" and "article" are goal words), leading to a wrong page. Same shared code and the same loss on Browser Use + WebFARM.	query construction
Europa EU countries, repeat 2	Repeated clicks on the same menu, a detour to a food.ec.europa.eu page, budget exhausted.	agent loop beyond recovery

What this means. None of the four losses is a wrong failure detection. Two are adapter-level interactions that a journal version can state as limitations (and the Debian case would be a completion under a neutral URL normalisation applied to all four systems); one is a known weakness of the shared query builder; one is an agent loop that the options could not break. Three pairs fail for every system: EPA recycle (repeat 1), Debian news (repeat 2), black hole (repeat 2).

Caveats for the paper. The runs were a week apart (web-v3 on 4–6 October, web-v4 on 11–13 October, web-v5 on 14 October); the tasks every system fails were the same in each week, which suggests little site drift. One infrastructure rerun in web-v5 (cPanel, repeat 1) and four in web-v4 (Coursera page load; Debian wall clock, see the run notes) are recorded in the plan files. The adapter differs from the Browser Use one in three places; the framework's model, gate, note text, option set, guard rule and memory are identical.

Summary for the supervisor

Our dataset trains any of three vision-language backbones to judge web-agent steps well (MCC 0.62–0.68), with Qwen2.5-VL-7B the best. On identical cases, the same model without training scores zero, so the gain is from our data, not the model. The trained 7B detector is in the same accuracy range as prompted GPT-4o reported elsewhere, at a tenth of the size. Attached to two different open-source agents with a 7B actor and no change to its model or settings, WebFARM raised task completion on 42 real-website tasks from 72.6 % to 85.7 % on Browser Use (84 paired episodes; 13 helped, 2 hurt; p = 0.007) and from 61.9 % to 82.1 % on SeeAct (21 helped, 4 hurt; p = 0.0009). It rescued 57 % and 66 % of the two agents' failures while harming 3 % and 8 % of their successes, and shortened episodes by 20 % and 32 %. The gain holds on 22 held-out tasks never seen during design (+9.1 and +22.7 points). With WebFARM the two agents are statistically indistinguishable (p = 0.65), and the weaker agent with WebFARM completes more tasks than the stronger agent without it. Published systems that add checking or recovery gain +4 to +8 points on one agent; small actors are known to be harmed by naive advice, which our redesigned recovery layer avoided. Remaining failures concentrate on pages whose target link shares no words with the goal, on misleading site searches, and on two adapter-level interactions on SeeAct; these are the subject of the error analysis that follows.

References: [1] arXiv:2509.14382 · [2] arXiv:2410.00689 · [3] arXiv:2505.20660 · [4] arXiv:2511.12997 · [5] arXiv:2405.16334 · [6] arXiv:2410.18963 · [7] arXiv:2504.07079 · [9] arXiv:2404.03648 · [10] arXiv:2406.01623 · [11] SeeAct, Zheng et al., ICML 2024, arXiv:2401.01614 (code: OSU-NLP-Group/SeeAct, package 0.2.9.0). Published numbers were transcribed from the papers' tables; please check them against the original PDFs before use in the thesis.